

Week 5. Problem set (solutions by Karim Khabibrakhmanov)

In a fictional multiplayer video game “Dragons and Pirates”, each player can select one of three character classes: a KNIGHT, a PIRATE, or a COOK. A team of 5 players wants to optimize their chances at the game by choosing the right combination of classes for each player in the team. When multiple players take the same class, their powers overlap, leading to an uneven increase in performance. For example, in the game multiple Knights cannot attack an enemy simultaneously, so overall performance metric for three Knights is less than three times the performance of a single Knight.

The table below gives the estimated performance for each character class for each possible number of players of the same class. Total performance of a team is the sum of performances for each class.

1. Describe a general algorithm that would find an optimal class assignments for any number P of players, any number C of classes, and table E with estimates:

- (a) Summarize the idea for a naïve recursive algorithm.

Answer: In a function, we recursively call C functions and so on P times. The recursion occurs without any optimizations, going through all possible class cases for each player. In the end, we simply return the maximum.

- (b) Clearly identify *overlapping subproblems* [CLRS, §14.3] (provide an explicit example for this problem).

Answer: Overlapping subproblems occur when the algorithm computes the same cases multiple times.

Let's say we have 2 classes ($C = 2$) and 4 players ($P = 4$):

- i. We take class 1 and class 2 and calculate the remaining case for 2 players

- ii. We take class 2 and class 1 and calculate the same case again

In the (2), we calculate the same case twice. This is overlapping subproblems.

- (c) Write down pseudocode for the dynamic programming [CLRS, §14] algorithm that solves the problem (top-down or bottom-up). It is enough to compute the optimal performance without keeping track of the exact class assignments.

Answer:

```
1 matrixForOverlap[P+1][C+1] = all element null
2 OptimalSolution(row, column):
3     if (row==0 or column==0)
4         return 0
5     if (matrixForOverlap[row][column] != null)
6         return matrixForOverlap[row][column]
7     max = 0
8     for i from 0 to row:
9         value = E[column - 1][i] + OptimalSolution(row - i, column - 1)
10        if value > max:
11            max = value
12
13    matrixForOverlap[row][column] = max
14    return matrixForOverlap[row][column]
```

2. Provide asymptotic worst-case time complexity with justification

- (a) for the naïve recursive algorithm

Answer: $\Theta(C^P)$

In a function, we recursively call C functions and so on P times.

- (b) for the dynamic programming algorithm

Answer: $\Theta(C \cdot P^2)$

The matrixForOverlap consists of $(P + 1) \cdot (C + 1)$ elements. For each table element, the function executes a loop that runs a maximum of *row* times. The maximum number of loop iterations is $O(P)$, since *row* can be no more than P . Therefore, the time complexity is $\Theta((C + 1) \cdot (P + 1) \cdot P) = \Theta(C \cdot P^2)$.

3. Apply the dynamic programming algorithm to an instance of the problem below with 3 classes and 6 players:

- (a) What is the best possible performance? What combination of classes gives that performance?

Answer:

We use (3 Cook, 2 Pirate and 1 Knight) or the option (3 Cook, 1 Pirate and 2 Knight) is also suitable. The maximum will be 70.

- (b) Provide the table with solutions used in the dynamic programming algorithm for subproblems that are computed in the algorithm.

Number of players	0	1	2	3	4	5	6
KNIGHT	0	12	23	33	42	55	62
PIRATE	0	13	24	33	40	45	50
COOK	0	11	22	34	43	52	61

Answer:

Number of players	0	1	2	3	4	5	6
0	0	0	0	0	0	0	0
KNIGHT	0	12	23	33	42	55	62
PIRATE	0	13	25	36	47	58	68
COOK	0	13	25	36	47	59	70

References

- [CLRS] Cormen, T.H., Leiserson, C.E., Rivest, R.L. and Stein, C., 2022. *Introduction to algorithms, Fourth Edition*. MIT press.